Product Requirements Document

Plivo Log Analysis Dashboard

Version: 1.0 **Date:** March 9, 2026 **Author:** Suraj Menon

---

1. Overview

1.1 Product Summary

The Plivo Log Analysis Dashboard is an internal tool for Plivo support agents to diagnose customer issues by analyzing voice (CDR) and SMS (MDR) logs. It automates the process of fetching logs from Redshift, running AI-powered analysis, and presenting findings through interactive charts, filterable tables, and exportable reports.

1.2 Problem Statement

Support agents currently spend significant time manually querying log databases, identifying error patterns, and correlating failures across carriers and regions. This tool reduces resolution time by automating log aggregation, error classification, trend detection, and generating actionable recommendations.

1.3 Target Users

User Type	Description
Plivo Support Agents	Primary users. Investigate customer complaints by analyzing CDR/MDR logs and pushing findings to Zendesk tickets.
Support Team Leads	Review PDF reports. Monitor health grades and error trends across customer accounts.

1.4 Key Metrics

- Time to diagnosis (target: under 2 minutes from form submission to actionable results)
- Analysis accuracy (AI summary relevance, correct error classification)
- Agent adoption rate (daily active sessions)

---

2. Architecture

2.1 Tech Stack

Layer	Technology	Details
Frontend	Next.js 16 (App Router), React 19, TypeScript	Single-page app with client-side state management
Styling	Tailwind CSS v4	CSS variable-based theming with dark mode support
Charts	Recharts	Interactive bar and area charts with drill-down
PDF Generation	jsPDF + html2canvas	Client-side PDF rendering
Backend	FastAPI (Python 3.9)	Async REST API with in-memory workflow state
AI Analysis	OpenAI API (via OpenRouter)	GPT-4o with structured JSON output; mock fallback

Data Source	Amazon Redshift (mocked)	CDR and MDR log tables; mock data generator for development
Ticketing	Zendesk API (mocked)	Push analysis summaries to support tickets

2.2 System Diagram

```
Browser (Next.js :3000)
  │
  ├── POST /api/trigger-analysis ──► FastAPI (:8000)
  │                                    ├── Mock Redshift → CDR/MDR records
  │                                    ├── Aggregation → error/carrier/region stats
  │                                    ├── OpenAI API → structured analysis JSON
  │                                    ├── Health Score computation
  │                                    └── Trend Detection (linear regression)
  │
  ├── GET /api/analysis-status/{id} ◄── Poll every 2s until completed
  │
  ├── POST /api/update-zendesk ───► Mock Zendesk API
  │
  └── Client-side: PDF generation, CSV/JSON export, session management
```

2.3 Data Flow

1. Agent submits analysis form (auth ID or email + date range + log type)
2. Backend creates an in-memory workflow and returns an `analysis_id`
3. Backend runs asynchronously: fetch logs → aggregate → AI analysis → compute health score + trend
4. Frontend polls `/api/analysis-status/{id}` every 2 seconds
5. On completion, frontend renders results dashboard with charts, table, and summary
6. Agent can drill down into charts, filter/export logs, generate PDF, or push to Zendesk

---

3. Functional Requirements

3.1 Analysis Form

ID	Requirement	Priority
F-101	Agent can enter a Plivo Auth ID and/or Customer Email as identifiers	P0
F-102	At least one identifier (Auth ID or Email) is required for submission	P0
F-103	Agent selects a date range (From Date, To Date)	P0
F-104	Agent selects log type: Voice (CDR) or SMS (MDR) via toggle buttons	P0
F-105	Submit button is disabled until validation passes	P0
F-106	Submit shows a loading spinner and disables re-submission	P1

3.2 Real-Time Progress Tracking

ID	Requirement	Priority
F-201	Four-step visual tracker: Queued → Fetching Logs → Analyzing → Complete	P0
F-202	Animated progress bar with percentage display	P0

F-203	Status message updates in real-time via polling	P0
F-204	Polling interval: 2 seconds	P0
F-205	Polling stops automatically when status is `completed` or `failed`	P0

3.3 Health Grade

ID	Requirement	Priority
F-301	System computes a health score (0-100) using weighted formula: 50% success rate + 30% error diversity + 20% trend	P0
F-302	Health score maps to letter grade: A (>=90), B (>=75), C (>=60), D (>=45), F (<45)	P0
F-303	Circular progress ring displays score with color-coded grade letter	P0
F-304	Descriptor text shown: Excellent, Good, Fair, Poor, Critical	P1
F-305	Score bar below the grade shows progress visually	P2

3.4 AI-Powered Analysis Summary

ID	Requirement	Priority
F-401	AI generates a 2-3 sentence executive summary of findings	P0
F-402	AI identifies and classifies issues by severity (critical, warning, info)	P0
F-403	Each issue includes: title, description, affected record count, and recommendation	P0
F-404	Issues are expandable/collapsible with a click	P1
F-405	Affected percentage is calculated and shown per issue	P1
F-406	Mock analysis fallback when AI API is unavailable (MOCK_AI=true)	P0

3.5 Trend Detection

ID	Requirement	Priority
F-501	System performs linear regression on daily error rates	P0
F-502	Trend direction classified: Stable (|slope| < 0.5), Increasing (slope > 0), Decreasing (slope < 0)	P0
F-503	Confidence level derived from R-squared: High (>=0.7), Medium (>=0.4), Low (<0.4)	P1
F-504	Trend indicator shown in Success Rate chart header with arrow icon and label	P0
F-505	Expandable tooltip shows detailed trend description	P2

3.6 Interactive Charts

ID	Requirement	Priority
F-601	Error Distribution: Horizontal bar chart showing top 10 error types	P0

F-602	Success Rate Over Time: Area chart with daily success rates (0-100%)	P0
F-603	Charts use theme-aware colors (light/dark mode)	P0
F-604	Error bars are clickable — clicking filters the log table to that error type	P1
F-605	Success rate dots are clickable — clicking filters the log table to that date	P1
F-606	Active drill-down dims non-selected chart elements for visual feedback	P1
F-607	Custom tooltips with full error names and formatted values	P1

3.7 Log Filtering & Search Table

ID	Requirement	Priority
F-701	Full-width interactive table displaying all raw log records	P0
F-702	Full-text search across all fields	P0
F-703	Dropdown filters for: Carrier, Region	P0
F-704	Type-specific filter: Hangup Cause (voice) or Status (SMS)	P0
F-705	Clickable column headers for ascending/descending sort	P1
F-706	Pagination with 20 rows per page and prev/next controls	P0
F-707	Drill-down integration: chart clicks pre-filter the table	P1
F-708	Drill-down filter bar shown with clear button when active	P1
F-709	CSV export of currently filtered record set	P0
F-710	JSON export of currently filtered record set	P0

3.8 PDF Report Export

ID	Requirement	Priority
F-801	"Download PDF Report" button renders a formatted A4 report	P0
F-802	PDF includes: Header (branding, date), Health Grade, Executive Summary, Key Metrics, Issues Table, Recommendations	P0
F-803	Multi-page support for long reports	P1
F-804	PDF generated client-side using html2canvas + jsPDF	P0
F-805	Loading spinner shown during generation	P1
F-806	Filename format: `plivo_analysis_report_YYYY-MM-DD.pdf`	P2

3.9 Zendesk Integration

ID	Requirement	Priority

F-901	Agent enters a Zendesk ticket ID	P0
F-902	Clicking "Update Ticket" pushes the analysis summary to the ticket	P0
F-903	Success/error message displayed after the API call	P0
F-904	Button disabled until ticket ID is provided; loading spinner while updating	P1

3.10 Dark Mode

ID	Requirement	Priority
F-1001	Toggle button in sidebar switches between light and dark themes	P0
F-1002	Theme preference persisted in localStorage across sessions	P0
F-1003	All components, charts, tooltips, and form inputs respect the active theme	P0
F-1004	CSS variable-based theming — no hardcoded colors in components	P0
F-1005	Sun/moon icon indicates current mode	P2

3.11 Session Management

ID	Requirement	Priority
F-1101	Analysis state persisted in sessionStorage (survives page refresh)	P0
F-1102	In-progress analysis resumes polling on reload	P0
F-1103	Completed results restored from session on reload	P0
F-1104	15-minute inactivity timeout when analysis is active	P0
F-1105	Timeout resets on user interaction (click, key, scroll, touch)	P0
F-1106	Timeout triggers a modal; confirming clears session and reloads	P0
F-1107	One-time privacy banner explains data handling and timeout policy	P2

---

4. API Specification

4.1 Endpoints

Method	Path	Description
`GET`	`/api/health`	Health check. Returns `{status: "healthy", version: "1.0.0"}`
`POST`	`/api/trigger-analysis`	Starts a new analysis workflow
`GET`	`/api/analysis-status/{analysis_id}`	Returns current progress, status, and results when complete
`POST`	`/api/update-zendesk`	Pushes analysis summary to a Zendesk ticket

4.2 Trigger Analysis

Request:

```
{
  "auth_id": "MAXXXXXXXXXXXXXXXXXX",   // optional, at least one of auth_id/email required
```

```
  "email": "customer@example.com",      // optional
  "from_date": "2026-03-01",            // required, YYYY-MM-DD
  "to_date": "2026-03-07",              // required, YYYY-MM-DD
  "log_type": "voice"                   // required, "voice" or "sms"
}
```

Response (202):

```
{
  "analysis_id": "a1b2c3d4-e5f6-...",
  "status": "queued",
  "message": "Analysis started"
}
```

4.3 Analysis Status

Response (200) — In Progress:

```
{
  "analysis_id": "a1b2c3d4-e5f6-...",
  "status": "analyzing",
  "progress_pct": 60,
  "message": "Running AI analysis on 174 records...",
  "result": null,
  "raw_logs": null
}
```

Response (200) — Completed:

```
{
  "analysis_id": "a1b2c3d4-e5f6-...",
  "status": "completed",
  "progress_pct": 100,
  "message": "Analysis complete",
  "result": {
    "summary": "Analysis of 174 voice records shows a 68.97% success rate...",
    "issues": [
      {
        "title": "High Rate of Invalid Number Format Errors",
        "severity": "critical",
        "description": "Error occurred 17 times across the analyzed period.",
        "affected_records": 17,
        "recommendation": "Investigate carrier routing for this error pattern."
      }
    ],
    "chart_data": {
      "error_distribution": [{"error": "4020 - Invalid Number Format", "count": 17}],
      "success_rate_over_time": [{"date": "2026-03-01", "rate": 72.5}]
    },
    "total_records": 174,
    "success_rate": 68.97,
    "date_range": "2026-03-01 to 2026-03-07",
    "log_type": "voice",
    "health_score": 54.5,
    "health_grade": "D",
```

```
    "trend": {
      "direction": "decreasing",
      "slope": -0.85,
      "confidence": "high",
      "description": "Error rates are trending downward, indicating an improving pattern."
    }
  },
  "raw_logs": [{"call_uuid": "...", "hangup_cause_code": 4020, "carrier": "AT&T", ...}]
}
```

4.4 Update Zendesk

Request:

```
{
  "ticket_id": "12345",
  "analysis_id": "a1b2c3d4-e5f6-...",
  "summary": "Analysis of 174 voice records shows..."
}
```

Response (200):

```
{
  "success": true,
  "ticket_id": "12345",
  "message": "Ticket 12345 updated with analysis summary. Internal note ID: a3f8c2..."
}
```

---

5. Data Models

5.1 CDR Record (Voice)

Field	Type	Description
`call_uuid`	string	Unique call identifier
`from_number`	string	Originating phone number
`to_number`	string	Destination phone number
`direction`	string	"inbound" or "outbound"
`duration_seconds`	int	Total call duration
`bill_duration_seconds`	int	Billable duration
`hangup_cause_code`	int	Hangup cause code (4000-5030)
`hangup_cause`	string	Human-readable cause
`initiation_time`	string	Call start timestamp
`answer_time`	string?	Answer timestamp (null if unanswered)
`end_time`	string	Call end timestamp
`carrier`	string	Carrier name

`region`	string	Geographic region
`sip_response_code`	int	SIP response code (200-503)

5.2 MDR Record (SMS)

Field	Type	Description
`message_uuid`	string	Unique message identifier
`from_number`	string	Sender number
`to_number`	string	Recipient number
`direction`	string	"inbound" or "outbound"
`message_type`	string	"sms"
`status`	string	"delivered", "failed", or "undelivered"
`error_code`	int?	Error code (null if successful)
`error_message`	string?	Error description
`sent_time`	string	Send timestamp
`delivered_time`	string?	Delivery timestamp (null if failed)
`carrier`	string	Carrier name
`region`	string	Geographic region
`units`	int	Message unit count

5.3 Health Score Formula

```
score = 0.50 * success_rate
      + 0.30 * error_diversity
      + 0.20 * trend_score

Where:
  error_diversity = max(0, 100 - unique_error_count * 10)
  trend_score     = 100 if stable/improving, penalized if worsening
                    max(0, 100 - slope * 20)

Grades:
  A >= 90 | B >= 75 | C >= 60 | D >= 45 | F < 45
```

5.4 Trend Detection Algorithm

```
Method: Ordinary Least Squares linear regression on daily error rates

Inputs: [error_rate_day1, error_rate_day2, ..., error_rate_dayN]

Outputs:
  slope     = regression slope (rate of change per day)
  r_squared = coefficient of determination (0-1)
```

```
Direction:
  |slope| < 0.5  → Stable
  slope > 0       → Increasing (worsening)
  slope < 0       → Decreasing (improving)

Confidence:
  r² >= 0.7  → High
  r² >= 0.4  → Medium
  r² < 0.4   → Low
```

---

6. Non-Functional Requirements

6.1 Performance

ID	Requirement
NF-101	End-to-end analysis completes within 10 seconds (mock mode)
NF-102	Frontend build produces optimized static output with no TypeScript errors
NF-103	Charts render responsively within their containers without overflow
NF-104	PDF generation completes within 5 seconds for typical reports
NF-105	Polling does not accumulate memory or duplicate requests

6.2 Usability

ID	Requirement
NF-201	All interactive elements have clear hover and disabled states
NF-202	Dark mode and light mode are visually consistent across all components
NF-203	Mobile-responsive layout: sidebar collapses to drawer on small screens
NF-204	Loading states shown for all async operations (submit, poll, Zendesk, PDF)
NF-205	Error messages are human-readable and displayed prominently

6.3 Reliability

ID	Requirement
NF-301	AI analysis falls back to mock result on API failure
NF-302	Session state survives page refresh via sessionStorage
NF-303	In-progress analysis resumes polling after refresh
NF-304	No data loss on theme toggle or navigation between phases

6.4 Security & Privacy

ID	Requirement
NF-401	No customer data persisted beyond the browser session
NF-402	Session auto-clears after 15 minutes of inactivity

NF-403	`Cache-Control: no-store` on all API responses
NF-404	CORS restricted to `http://localhost:3000`
NF-405	Privacy banner informs agents about data handling

---

7. UI Layout

7.1 Page Structure

```
┌──────────┬──────────────────────────────────────────┐
│          │  Header (breadcrumb, title, subtitle)    │
│  Sidebar │──────────────────────────────────────────│
│          │                                          │
│  Logo    │  [Privacy Banner]                        │
│  Nav     │                                          │
│  ------  │  Phase: Form                             │
│  Theme   │  ┌──────────────────────────┐            │
│  Toggle  │  │  Analysis Form (centered)  │            │
│  ------  │  │  Auth ID / Email / Dates   │            │
│  User    │  │  Log Type Toggle / Submit  │            │
│  Profile │  └──────────────────────────┘            │
│          │                                          │
│          │  Phase: Processing                       │
│          │  ┌──────────────────────────┐            │
│          │  │  Status Tracker            │            │
│          │  │  Steps + Progress Bar      │            │
│          │  └──────────────────────────┘            │
│          │                                          │
│          │  Phase: Results                          │
│          │  ┌──────────────────────────────────┐    │
│          │  │ Health Grade [Ring] [Score] [PDF Button] │
│          │  └──────────────────────────────────┘    │
│          │                                          │
│          │  ┌──────────────────────────────────┐    │
│          │  │ AI Summary + Expandable Issues     │    │
│          │  └──────────────────────────────────┘    │
│          │                                          │
│          │  ┌──────────────┐  ┌──────────────────┐  │
│          │  │ Error Dist.    │  │ Success Rate + Trend│
│          │  │ (Bar Chart)    │  │ (Area Chart)       │
│          │  └──────────────┘  └──────────────────┘  │
│          │                                          │
│          │  ┌──────────────────────────────────┐    │
│          │  │ Log Table (full width)             │    │
│          │  │ Search | Filters | Sort | Paginate │    │
│          │  └──────────────────────────────────┘    │
│          │                                          │
│          │  ┌──────────────────────────────────┐    │
│          │  │ Zendesk Integration                │    │
│          │  └──────────────────────────────────┘    │
│          │              [Start New Analysis]        │
└──────────┴──────────────────────────────────────────┘
```

7.2 Theme Variables

Variable	Light	Dark
`--background`	`#f8f9fa`	`#0f1117`

`--foreground`	`#111827`	`#e5e7eb`
`--card-bg`	`#ffffff`	`#1a1b23`
`--card-border`	`#e5e7eb`	`#2d2e3a`
`--success`	`#16a34a`	`#22c55e`
`--danger`	`#dc2626`	`#ef4444`
`--warning`	`#d97706`	`#f59e0b`
`--info`	`#3b82f6`	`#60a5fa`

---

8. Configuration

8.1 Environment Variables

Variable	Default	Description
`OPENAI_API_KEY`	(empty)	API key for OpenRouter/OpenAI
`OPENAI_BASE_URL`	`https://openrouter.ai/api/v1`	AI API base URL
`OPENAI_MODEL`	`openai/gpt-4o`	Model identifier
`MOCK_AI`	`true`	Use mock analysis (no API calls)
`ZENDESK_SUBDOMAIN`	(empty)	Zendesk subdomain
`ZENDESK_EMAIL`	(empty)	Zendesk agent email
`ZENDESK_API_TOKEN`	(empty)	Zendesk API token
`CORS_ORIGINS`	`["http://localhost:3000"]`	Allowed CORS origins
`LOG_LEVEL`	`INFO`	Backend log level
`NEXT_PUBLIC_API_BASE_URL`	`http://localhost:8000`	Backend URL for frontend

8.2 Frontend Constants

Constant	Value	Description
`POLLING_INTERVAL_MS`	`2000`	Status poll frequency
`SESSION_TIMEOUT_MS`	`900000`	Inactivity timeout (15 min)
`SESSION_STORAGE_KEY`	`plivo_analysis_state`	sessionStorage key

---

9. File Structure

```
LogAnalyserProject/
├── backend/
│   └── app/
│       ├── main.py                        # FastAPI app, middleware, routers
│       ├── config.py                      # Settings (env vars)
│       ├── api/
```

```
│       │   ├── health.py                    # GET /api/health
│       │   ├── analysis.py                  # POST trigger + GET status
│       │   └── zendesk.py                   # POST update-zendesk
│       ├── core/
│       │   ├── constants.py                 # Hangup codes, carriers, regions
│       │   └── exceptions.py                # Custom exceptions
│       ├── models/
│       │   ├── analysis.py                  # AnalysisResult, TrendInfo, ChartData
│       │   ├── cdr.py                       # CDRRecord
│       │   ├── mdr.py                       # MDRRecord
│       │   ├── requests.py                  # TriggerAnalysisRequest
│       │   └── responses.py                 # API response models
│       └── services/
│           ├── workflow_manager.py           # Workflow orchestration
│           ├── mock_redshift.py             # Mock CDR/MDR data generation
│           ├── openai_analyzer.py           # AI analysis + aggregation
│           ├── health_score.py              # Health score computation
│           ├── trend_detection.py           # Linear regression trend detection
│           └── mock_zendesk.py              # Mock Zendesk API
│
└── frontend/
    └── src/
        ├── app/
        │   ├── globals.css                  # CSS variables, dark mode, animations
        │   ├── layout.tsx                   # Root layout (Inter font, metadata)
        │   └── page.tsx                     # Main page (3-phase workflow)
        ├── components/
        │   ├── AnalysisForm.tsx             # Input form
        │   ├── StatusTracker.tsx            # Progress stepper
        │   ├── HealthGradeCard.tsx          # Score ring + grade
        │   ├── AISummaryCard.tsx            # Summary + expandable issues
        │   ├── ErrorDistributionChart.tsx   # Bar chart (drill-down)
        │   ├── SuccessRateChart.tsx         # Area chart (trend + drill-down)
        │   ├── TrendIndicator.tsx           # Arrow + confidence badge
        │   ├── LogTable.tsx                 # Filterable/sortable table
        │   ├── ZendeskUpdateButton.tsx      # Ticket integration
        │   ├── PdfReportExport.tsx          # PDF generation button
        │   ├── PdfReportContent.tsx         # PDF layout (inline styles)
        │   ├── Sidebar.tsx                  # Navigation + theme toggle
        │   ├── PrivacyBanner.tsx            # One-time info banner
        │   ├── SessionTimeoutModal.tsx      # Timeout dialog
        │   └── RawLogsDownload.tsx          # Legacy download (replaced by LogTable)
        ├── hooks/
        │   ├── useAnalysis.ts              # Workflow state + session management
        │   ├── usePolling.ts               # Configurable interval polling
        │   ├── useSessionTimeout.ts        # Inactivity detection
        │   ├── useTheme.ts                 # Dark/light toggle
        │   └── useChartColors.ts           # CSS variable reader for charts
        ├── lib/
        │   ├── api.ts                      # API client (fetch wrappers)
        │   ├── types.ts                    # TypeScript interfaces
        │   ├── session.ts                  # sessionStorage helpers
        │   └── download.ts                 # CSV/JSON export utilities
        └── constants/
            └── index.ts                    # Polling interval, timeout, API URL
```

---

10. Verification Checklist

#	Feature	Test
1	**Analysis Form**	Submit with auth ID only, email only, both. Verify validation blocks empty submissions.
2	**Progress Tracking**	Trigger analysis, observe 4 steps animate through. Verify progress bar reaches 100%.
3	**Health Grade**	Complete analysis, verify grade badge shows correct letter (A-F) with matching color.
4	**AI Summary**	Verify summary text, issue cards with severity badges, expandable details, recommendations.
5	**Trend Detection**	Check Success Rate chart header shows trend arrow (up/down/stable) with confidence badge.
6	**Error Distribution Chart**	Verify bars render for top errors. Click a bar → log table filters to that error.
7	**Success Rate Chart**	Verify area chart with gradient. Click a dot → log table filters to that date.
8	**Log Table**	Test search, carrier filter, region filter, type-specific filter. Sort columns. Paginate.
9	**Drill-down**	Click error bar → filter bar appears in table. Click date dot → filter bar appears. Clear button works.
10	**CSV/JSON Export**	Export filtered logs as CSV and JSON. Verify file contents match displayed data.
11	**PDF Report**	Click Download PDF. Verify PDF contains header, grade, summary, metrics, issues, recommendations.
12	**Zendesk**	Enter ticket ID, click Update. Verify success message displayed.
13	**Dark Mode**	Toggle in sidebar. Verify all components, charts, tooltips switch. Refresh → theme persists.
14	**Session Persistence**	Submit analysis, refresh page during processing → polling resumes. Refresh on results → results restored.
15	**Session Timeout**	Wait 15 min (or reduce timer for testing). Verify modal appears. Confirm → session cleared.
16	**Responsive**	Resize to mobile width. Sidebar collapses to hamburger menu. Table scrolls horizontally.
17	**Backward Compat**	Existing polling, session persistence, and Zendesk integration still work after all changes.